

ICS-Lab0: GitLab实验报告

24300810019@m.fudan.edu.cn 吴世强

仓库地址: <https://github.com/garyagasa/ICS-GitLab>

1. 认真阅读文档，学习 Git 的基本用法，并在报告中回答文档中的问题。（15分）

- 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

答：之前基本以独立开发为主，没有过真正意义上的多人协同开发经历；以往小组合作时主要是各自写好后通过聊天软件或网盘互传文件、手动合并，常出现版本混乱、互相覆盖的问题。通过本实验系统学习 Git，希望掌握分支管理、合并与冲突解决等多人协作所需的技能。

- 思考一下，Git 为什么要设计“暂存-提交”两个步骤？

答：Git 把一次提交拆成“暂存（git add）”和“提交（git commit）”两步，主要好处有：

- 选择性提交**：可只把部分改动加入暂存区、其余留在工作区，让每次提交聚焦一个主题、更“原子”；
- 提交前可检查**：暂存后可用 `git diff --cached` 查看即将提交的内容，确认无误再提交；
- 细粒度控制**：`git add -p` 甚至能只暂存文件里的部分代码块（hunk）；
- 工作区与版本库解耦**：工作区里未完成的中间状态不会被被动记录，只有主动 add 的内容才会进入提交。简言之，暂存区决定“这次提交什么”，提交步骤才真正把快照记录下来。

- `git branch` 和 `git branch -a` 的区别是什么？查阅资料并回答。

```
(base) gary@LAPTOP-1P7N3FS9:/mnt/c/Users/wsqa/Desktop/26Fall/ics/ICS-GitLab$ git branch
dev
feature
* main
(base) gary@LAPTOP-1P7N3FS9:/mnt/c/Users/wsqa/Desktop/26Fall/ics/ICS-GitLab$ git branch -a
dev
feature
* main
remotes/origin/HEAD -> origin/main
remotes/origin/main
```

答：`git branch` 只列出本地分支；`git branch -a`（`--all`）列出本地分支 + 远程跟踪分支（`remotes/origin/...`）。远程跟踪分支是 Git 在本地保存的远程仓库状态镜像，不联网也能查看（`git branch -r` 则只列出远程跟踪分支）。下面的输出直观展示了区别：

代码块

```
1 (base) gary@LAPTOP-1P7N3FS9:/mnt/c/Users/wsqa/Desktop/26Fall/ics/ICS-
  GitLab$ git branch
2 * main
3 (base) gary@LAPTOP-1P7N3FS9:/mnt/c/Users/wsqa/Desktop/26Fall/ics/ICS-
  GitLab$ git branch dev
4 (base) gary@LAPTOP-1P7N3FS9:/mnt/c/Users/wsqa/Desktop/26Fall/ics/ICS-
  GitLab$ git branch -a
5 dev
6 * main
7 remotes/origin/HEAD -> origin/main
8 remotes/origin/main
```

2. 使用此仓库建立个人仓库，完成 `main.c` 文件中的 `TODO` 部分并进行一次 `commit`。（50 分）

代码块

```
1 printf("Welcome to Fudan NLP Group\n");
```

- 只要填入任意字符串就算完成，当然你也可以随意发挥（程序的正确性不纳入计分，有修改即可）。
- 如果你想要编译运行 `main.c`，执行以下命令

代码块

```
1 make
2 ./main
3 make clean
```

```
(base) gary@LAPTOP-1P7N3FS9:/mnt/c/Users/wsqa/Desktop/26Fall/ics/ICS-GitLab$ make
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
(base) gary@LAPTOP-1P7N3FS9:/mnt/c/Users/wsqa/Desktop/26Fall/ics/ICS-GitLab$ ./main
Hello from main and feature branch (conflict resolved)
```

代码块

```
1 git add main.c
2 git commit -m "revise the output string"
```

提交后历史如下（`git log --oneline`）：

代码块

```
1 298beb4 revise the output string
2 aa2f128 Initial commit
```

3. 在下面的三个网页中任选其二进行阅读，简要概括其内容，并谈谈你对“为什么要学习 Git”这个问题的理解。（15 分）

- [Commit Message 规范](#)
- [Git Flow 分支控制](#)
- [语义化版本](#)

3.1 Commit Message 规范

该文推荐 **Angular 提交信息规范**，把一条 commit message 分成三部分：

代码块

```
1 <type>(<scope>): <subject>
2
3 <body>
4
5 <footer>
```

其中 Header (`type(scope): subject`) 必填，**subject 用动词开头、一般现在时、首字母小写、不加句号，不超过 50 个字符**；**Body 说明改动动机与之前行为的对比**；**Footer 只用于标记破坏性变更 (`BREAKING CHANGE:`) 和关闭 issue (`Closes #234`)**。常用的 7 种 `type` 为：`feat`（新功能）、`fix`（修 bug）、`docs`（文档）、`style`（格式）、`refactor`（重构）、`test`（测试）、`chore`（构建/工具）。

核心价值在于：当所有提交都遵循同一格式时，可以用 `conventional-changelog` 等工具**自动生成 Change Log**（其中 `feat`、`fix` 一定会出现在日志里），并可用 `git log -grep` 精确筛选某类提交，而不必手工维护发布说明。

3.2 语义化版本 (SemVer)

语义化版本采用 `主版本号.次版本号.修订号`（`MAJOR.MINOR.PATCH`，如 `1.2.3`）的格式，每个数字都是非负整数且禁止补零：

- **主版本号 MAJOR**：做了**不兼容的 API 修改**时递增（递增时次版本号、修订号归零）；
- **次版本号 MINOR**：做了**向下兼容的功能新增**时递增（递增时修订号归零），API 被标记弃用也算；
- **修订号 PATCH**：做了**向下兼容的问题修正**时递增。

`0.y.z` 表示开发初始阶段，API 不稳定，可随时变更；`1.0.0` 标志公共 API 正式形成。已发布的版本内容不可更改，任何修改都必须以新版本号重新发布；版本号后可附加先行版本号

(如 `-alpha`、`-rc.1`) 和编译信息 (如 `+20130313144700`)。

3.3 对"为什么要学习 Git"的理解

Git 解决的问题，本质上是**多人协作下的代码版本管理**：它用"快照 + 分支"的模型，让每个人可以在自己的分支上独立开发而不互相干扰，再通过 merge 把成果合并；出现冲突时也有明确的机制去发现和解决。配合规范化的 commit message 和语义化版本，还能让历史可追溯、发布可自动化。因此，学会 Git 是进入任何真实团队开发的必备基础，它能显著减少"传文件、互相覆盖、找不到旧版本"这类低效和事故。

4. 学习 Git 分支管理，新建 `feature` 分支，在该分支以及 `main` 分支上对 `main.c` 分别进行一次修改与提交（10 分）。随后将 `feature` 分支 merge 到 `main` 分支（即切换回 main 分支执行 `git merge feature`），并处理发生的合并冲突（10 分）。

- 在两个分支上的提交必须要满足：在 `main` 分支合并时会出现冲突。请你解决这个冲突，并在实验报告里截图表明你遇到并解决了冲突。
- 请阅读 `git merge` 部分，思考如何修改 `main.c` 会出现冲突。
- 如果你两次提交之后合并没有出现冲突，不必担心，你可以不用撤回之前的提交，而是继续尝试提交修改并 merge，直到出现冲突并解决。

a. 新建并切换到 `feature` 分支：

代码块

```
1 git switch -c feature
```

b. 在 `feature` 分支上修改 `main.c`，把输出改为 `Hello from feature branch`，然后提交：

代码块

```
1 git add main.c
2 git commit -m "feat: print message from feature branch"
```

c. 切回 `main` 分支，把同一行输出改为 `Hello from main branch`（刻意改成不同内容以便冲突），然后提交：

代码块

```
1 git switch main
2 # 修改 main.c
```

```
3 git add main.c
4 git commit -m "feat: print message from main branch"
```

d. 在 `main` 分支上执行合并，触发冲突：

代码块

```
1 git merge feature
```

代码块

```
1 Auto-merging main.c
2 CONFLICT (content): Merge conflict in main.c
3 Automatic merge failed; fix conflicts and then commit the result.
```

代码块

```
1 <<<<<<< HEAD
2     printf("Hello from main branch\n");
3     =====
4     printf("Hello from feature branch\n");
5 >>>>>>> feature
```

e. 解决冲突：手动编辑 `main.c`，删除冲突标记，保留最终想要的内容：

代码块

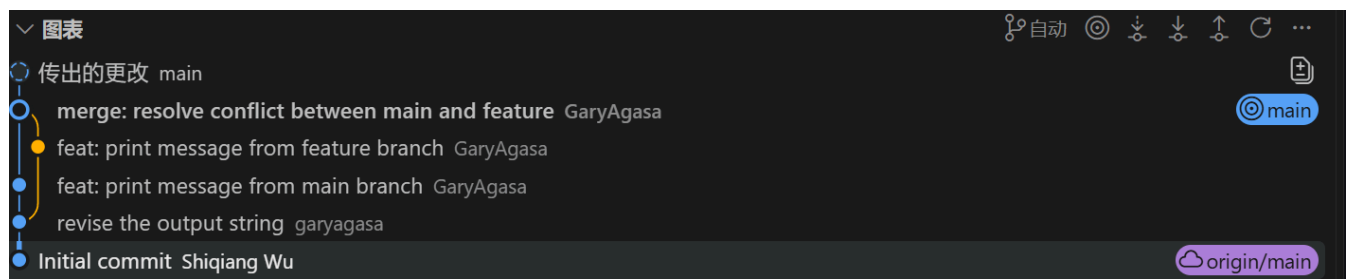
```
1 printf("Hello from main and feature branch (conflict resolved)\n");
```

f. 标记已解决并提交合并：

代码块

```
1 git add main.c
2 git commit -m "merge: resolve conflict between main and feature"
```

合并后的完整提交历史（`git log --oneline --graph --all`）：



代码块

```
1 * f1dee73 merge: resolve conflict between main and feature
2 |\
3 | * 0c1b31c feat: print message from feature branch
4 * | 5ce892e feat: print message from main branch
5 |/\
6 * 298beb4 revise the output string
7 * aa2f128 Initial commit
```

5.在 `main` 分支提交一份实验报告（实验报告单独评分），格式要求为 `PDF` 或 `Markdown`。内容包括：

- 文档中要求回答的问题
- 你的实验步骤
- 必要的截图
- 你的建议（可选）